

A PRODUCTION CURRICULUM

# Building RAG on AWS

---

*Managed, custom, agentic, and graph-based retrieval — the full design space, the decision framework, and a step-by-step path from a managed baseline to a production capstone.*

Third in a series with **The Comprehensive RAG Study Guide** and **The Comprehensive LLM Study Guide**

Architecture • AWS services • Trade-offs • Hands-on builds • Evaluation

Practitioner level — assumes RAG fundamentals

# Table of Contents

---

## Phase 0 — Foundations & the Decision Framework

0.1 The design space • 0.2 The five patterns • 0.3 Control vs. managed • 0.4 Cost, latency, scale • 0.5 Three reference architectures • 0.6 How to choose

## Phase 1 — The Managed Baseline: Bedrock Knowledge Bases

1.1 Two KB products • 1.2 Data sources & ingestion • 1.3 Chunking strategies • 1.4 Vector-store choices • 1.5 Retrieve & RetrieveAndGenerate • 1.6 Filtering, hybrid, reranking • 1.7 Guardrails • Build

## Phase 2 — The Retrieval Core, Decomposed

2.1 Embeddings • 2.2 Vector stores on AWS • 2.3 ANN indexing & HNSW • 2.4 Hybrid search • 2.5 Reranking • 2.6 Query understanding • Build

## Phase 3 — Custom RAG on ECS

3.1 Why go custom • 3.2 The ingestion pipeline • 3.3 Parsing & OCR • 3.4 The service topology • 3.5 Fargate, SQS, Step Functions • 3.6 Serving & scaling • 3.7 IaC • Build

## Phase 4 — Agentic RAG

4.1 When retrieval becomes a tool • 4.2 The patterns (router, planner, CRAG, Self-RAG) • 4.3 The AWS agent stack • 4.4 Strands & AgentCore • 4.5 Tools via MCP • 4.6 Memory & multi-agent • Build

## Phase 5 — GraphRAG: Knowledge Graph as Source

5.1 When graphs win • 5.2 Managed GraphRAG • 5.3 Custom graph construction • 5.4 Neptune & traversal • 5.5 Hybrid graph + vector • Build

## Phase 6 — Structured & Multimodal RAG

6.1 Text-to-SQL • 6.2 When docs are tables • 6.3 Multimodal parsing • 6.4 Federating structured + unstructured • Build

## Phase 7 — Production Hardening

7.1 Evaluation (retrieval + generation) • 7.2 Continuous eval in CI/CD • 7.3 Observability & cost • 7.4 Security & multi-tenancy • 7.5 Caching • 7.6 Latency & scaling • Build

## Phase 8 — Capstone & Reference Material

8.1 The capstone brief • 8.2 A worked decision • 8.3 Service cheat-sheet • 8.4 Glossary

## PHASE 0

# Foundations & the Decision Framework

Before any service, one skill: given a set of production requirements, choose the right RAG architecture and justify it. Every later phase teaches one option; this phase teaches how to pick among them. Read it first, and return to it before the capstone.

## 0.1 The design space is a spectrum, not a menu

"How should I build RAG on AWS?" has no single answer because the question hides three independent axes. Locate your problem on all three and the architecture falls out:

- **Control axis — managed ↔ custom.** How much of the pipeline you hand to AWS versus own yourself. This is the dominant axis and the one that most affects team effort, cost curve, and flexibility.
- **Source axis — unstructured ↔ structured ↔ graph.** Are your "documents" prose (PDFs, wikis), tables (warehouses), relationships (entities and links), or a mix? The source shape dictates the retrieval mechanism.
- **Query axis — single-shot ↔ multi-hop.** Can a question be answered by one retrieve-then-generate pass, or does it need decomposition, several retrievals, and reasoning across them? This axis decides whether you need an agent.

### THE CORE DISCIPLINE

Production RAG is not about picking the most advanced pattern — it is about picking the *least* complex pattern that meets the requirement. Complexity is a cost you pay forever in latency, dollars, and debugging. Escalate only when a concrete requirement forces you up the ladder.

## 0.2 The five patterns

Five recurring architectures cover almost everything. They compose — the sophisticated systems in Phase 8 are combinations — but you must understand each in isolation first.

| Pattern                        | One-line essence                                 | Retrieval shape               | Phase |
|--------------------------------|--------------------------------------------------|-------------------------------|-------|
| <b>Managed KB</b>              | AWS owns ingestion → retrieval; you call an API  | Vector (optionally hybrid)    | 1     |
| <b>Custom on ECS</b>           | You own and tune every stage                     | Whatever you build            | 3     |
| <b>Agentic RAG</b>             | The model decides when/how to retrieve, iterates | Retrieval-as-tool, multi-step | 4     |
| <b>GraphRAG</b>                | Retrieve over entities & relationships           | Graph traversal + vector      | 5     |
| <b>Structured / multimodal</b> | Query tables (text-to-SQL) or media              | SQL / vision parsing          | 6     |

### 0.3 The control axis, in detail

Everything hinges on how much you hand to AWS. Picture the same seven-stage pipeline — **parse** → **chunk** → **embed** → **store** → **retrieve** → **rerank** → **generate** — and note who owns each stage under each option:

| stage:       | PARSE   | CHUNK   | EMBED   | STORE   | RETRIEVE | RERANK  | GENERATE |                  |
|--------------|---------|---------|---------|---------|----------|---------|----------|------------------|
| Managed KB   | [ AWS ] | [ AWS ] | [ AWS ] | [ AWS ] | [ AWS ]  | [ AWS ] | [ AWS ]  | ← call one API   |
| Cust.-mgd KB | [ AWS ] | [ you ] | [ you ] | [ you ] | [ AWS ]  | [ you ] | [ you ]  | ← you pick knobs |
| Custom ECS   | [ you ] | [ you ] | [ you ] | [ you ] | [ you ]  | [ you ] | [ you ]  | ← you own it all |

Note the important nuance from Phase 1: Bedrock now ships **two** Knowledge Base products — a fully managed one (AWS owns even the vector store and reranker) and the original customer-managed one (you choose store, embedding model, and chunking, and call retrieval APIs). "Managed" is itself a small spectrum.

### 0.4 The three forces: cost, latency, scale

Requirements usually reduce to trade-offs among three forces plus flexibility. Keep this table in mind as the tie-breaker whenever two architectures both "work":

| Force                       | Favours managed when...                             | Favours custom when...                                         |
|-----------------------------|-----------------------------------------------------|----------------------------------------------------------------|
| <b>Team effort</b>          | Small team, ship fast, standard docs                | You have platform engineers and time                           |
| <b>Cost at scale</b>        | Low/moderate volume; managed overhead is negligible | High, sustained volume where per-call managed markup dominates |
| <b>Latency</b>              | Default budgets are fine                            | You need tight tail-latency control, caching, co-location      |
| <b>Flexibility</b>          | Standard chunking/models suffice                    | Custom parsing, self-hosted models, bespoke retrieval logic    |
| <b>Control / compliance</b> | AWS-managed isolation is acceptable                 | Strict tenancy, residency, or audit requirements               |

### 0.5 Three reference architectures

Most systems are a variation on one of these three. Learn them as templates; the phases fill in the detail.

#### A — Managed baseline (ship in days)

```
S3 (docs) → Bedrock Knowledge Base → RetrieveAndGenerate → API Gateway + Lambda → app
              (chunk+embed+store+retrieve managed)           |
                                                              Bedrock Evaluations (RAG eval)
```

Fastest path to a grounded assistant. Almost no infrastructure. The right default, and often the right *permanent* answer for internal tools and moderate volume.

## B — Custom pipeline on ECS (control & scale)

```
upload → S3 → event → SQS → Fargate INGESTION workers
      (Textract/parse → chunk → Bedrock embed) → OpenSearch (hybrid, HNSW)
query → API GW → Fargate RETRIEVAL svc → (hybrid → rerank) → Bedrock generate → response
orchestrate: Step Functions • OTel→CloudWatch tracing • cost dashboard • CI/CD eval gate
```

You own parsing, chunking, indexing, and retrieval logic. This is the architecture for demanding, high-volume, or compliance-bound systems — and the shape most in-house platform teams converge on.

## C — Agentic + GraphRAG (reasoning over heterogeneous sources)

```
user → Agent (Strands on AgentCore Runtime)
      | plan → choose tool(s) → observe → reflect → answer (CRAG / Self-RAG loop)
      |— tool: vector KB (unstructured docs)
      |— tool: Neptune graph traversal (relationships)
      |— tool: text-to-SQL (warehouse tables)
      |— tool: web search
AgentCore: Memory • Identity • Observability • Gateway(MCP)
```

The endgame for questions that need multi-hop reasoning across mixed sources. Powerful and expensive; adopt deliberately, only after a simpler baseline demonstrably falls short.

## 0.6 The decision framework

Walk this ladder top to bottom. Stop at the first rung that meets your requirements — do not climb higher for its own sake.

- |                                                                                     |                                                 |
|-------------------------------------------------------------------------------------|-------------------------------------------------|
| 1. Standard docs, single-shot Q&A, moderate volume?                                 | → Managed KB (Phase 1). STOP.                   |
| 2. Need custom chunking / self-hosted models / hybrid logic, but still single-shot? | → Customer-managed KB / Custom ECS (Phase 3).   |
| 3. High volume, strict tenancy/residency, low latency?                              | → Custom ECS (Phase 3).                         |
| 4. Multi-hop / needs tools beyond retrieval / spans heterogeneous sources?          | → add an Agentic layer (Phase 4).               |
| 5. About relationships/connections, multi-hop across linked entities?               | → add GraphRAG (Phase 5).                       |
| 6. "Documents" are tables or media?                                                 | → add Structured / Multimodal (Phase 6).        |
| — in all cases —————→                                                               | harden: eval, observability, security (Phase 7) |

### PHASE 0 TAKEAWAYS

- Place every problem on three axes: **control**, **source shape**, **query complexity**.
- Pick the **least complex** pattern that meets the requirement; complexity is a permanent cost.
- Five patterns cover the space and **compose**: managed KB, custom ECS, agentic, graph, structured/multimodal.
- Three reference architectures — managed baseline, custom ECS, agentic+graph — are your templates.
- Use the **decision ladder**: escalate only when a concrete requirement forces you up.

---

## PHASE 1

# The Managed Baseline: Bedrock Knowledge Bases

Start here for every project, even ones you expect to outgrow — a managed baseline is the fastest way to a working system and the yardstick every custom build must beat. This phase makes you fluent in Amazon Bedrock Knowledge Bases (KB).

## 1.1 Two Knowledge Base products — know which you're using

Bedrock now offers two KB products, and confusing them wastes days:

- **Customer-managed Knowledge Bases** (the original, GA since 2023). You choose the vector store, the embedding model, and the chunking strategy, and you call the retrieval APIs directly. This is the product that exposes every tuning knob in this guide. Use it whenever you want control while still letting AWS run the ingestion and retrieval plumbing.
- **Amazon Bedrock Managed Knowledge Base** (newer, more fully managed). AWS manages the vector store, embeddings, and reranker for you — the closest thing to "RAG as a single API." Use it for the fastest possible baseline when you don't need to tune internals.

The rest of this phase assumes the customer-managed product, because its knobs are what you must understand to make the Phase 3 build/buy decision later.

## 1.2 Data sources & the ingestion job

A KB connects to a **data source** and runs an **ingestion job** that parses, chunks, embeds, and indexes your content — no pipeline code. Beyond S3, connectors exist for web crawlers, Confluence, SharePoint, and Salesforce. Re-running ingestion syncs changes. Two practitioner notes: ingestion is asynchronous and can take a while on large corpora, and **metadata** you attach to documents (via sidecar JSON) becomes filterable at query time — design your metadata schema up front, because it is your main lever for access control and relevance scoping.

## 1.3 Chunking strategies

Chunking decides what a "unit of retrieval" is, and it is the single highest-leverage KB setting. Available strategies:

| Strategy               | What it does                                                        | Use when                                             |
|------------------------|---------------------------------------------------------------------|------------------------------------------------------|
| <b>Fixed-size</b>      | Splits every N tokens with overlap                                  | Uniform prose; simplest baseline                     |
| <b>Semantic</b>        | Splits at meaning boundaries (embedding-similarity breaks)          | Mixed-topic docs where clean boundaries matter       |
| <b>Hierarchical</b>    | Small child chunks for matching, larger parent returned for context | Long structured docs; best recall-vs-context balance |
| <b>Custom (Lambda)</b> | Your own function does the splitting                                | Domain formats (code, transcripts, tables)           |

### PRACTITIONER VERDICT

Start with hierarchical for real documents — it retrieves precisely (small children) but hands the model enough surrounding context (large parents), which is usually what fixes "the answer was in the doc but the model didn't see enough." Fall back to a custom Lambda chunker the moment your content has structure a generic splitter destroys (code blocks, Q&A pairs, tabular rows).

## 1.4 Vector-store choices

The customer-managed KB lets you back retrieval with any of a wide set of stores, and the choice has real cost and operational consequences:

| Store                                   | Character                                     | Pick when                                             |
|-----------------------------------------|-----------------------------------------------|-------------------------------------------------------|
| <b>S3 Vectors</b>                       | New, cheapest, serverless, higher latency     | Large corpora, cost-sensitive, latency-tolerant       |
| <b>OpenSearch Serverless</b>            | Default managed vector + keyword; scales well | Most production KBs; you want hybrid                  |
| <b>Aurora PostgreSQL (pgvector)</b>     | Vectors alongside relational data             | You already run Postgres; want SQL + vectors together |
| <b>Neptune Analytics</b>                | Graph + vector (powers GraphRAG)              | Relationship-heavy data (Phase 5)                     |
| <b>Pinecone / Redis / MongoDB Atlas</b> | Third-party managed                           | Existing investment or specific feature need          |

For a first production system, **OpenSearch Serverless** is the safe default; consider **S3 Vectors** when the corpus is large and a little extra latency is acceptable in exchange for a much lower bill.

## 1.5 Retrieve vs. RetrieveAndGenerate

The KB exposes two APIs, and choosing between them is an architecture decision, not a detail:

- **Retrieve** returns the top-k chunks and their metadata — nothing more. You then build your own prompt and call the model yourself. Choose this when you want to own prompt construction, inject business logic, blend KB results with other sources, or control the generation model and parameters. It is the seam you will widen in Phase 3.
- **RetrieveAndGenerate** does the whole loop — retrieve, assemble a prompt, call the model, and return a cited answer. Choose this for the fastest baseline and when the default prompt behaviour is good enough.

### AWS SERVICES IN PLAY

**Amazon Bedrock Knowledge Bases** (orchestration), **Amazon S3** (source), a **vector store** (above), a Bedrock **embedding model** (e.g. Titan Text Embeddings, Cohere Embed) for indexing, and a Bedrock **foundation model** for generation. Wrap it with **API Gateway + Lambda** for a public endpoint, and **IAM** to scope who can query.

## 1.6 Metadata filtering, hybrid search, and reranking

Three relevance levers turn a demo into something trustworthy:

- **Metadata filtering.** Restrict retrieval by attributes you attached at ingestion (tenant, department, date, document type). This is both a relevance tool and your primary **access-control** mechanism in multi-tenant systems — filter before you retrieve, never after.
- **Hybrid search.** Combine dense vector similarity with sparse keyword (BM25) matching. Vector search captures meaning; keyword search captures exact terms, codes, and names that embeddings blur. Hybrid reliably beats either alone on real corpora; enable it when your store supports it (OpenSearch).
- **Reranking.** After retrieving a generous candidate set, a cross-encoder reranker (e.g. Cohere Rerank 3.5, Amazon Rerank) reorders them by true relevance to the query before you spend context tokens on them. Retrieve wide (say top-50), rerank, keep the best few. This is one of the cheapest large wins in RAG quality.

## 1.7 Guardrails

**Amazon Bedrock Guardrails** attach to the KB to filter harmful content, block denied topics, redact or block PII, and reduce hallucination via contextual-grounding checks that verify the answer is actually supported by the retrieved passages. In regulated settings this is not optional; wire it in from the baseline rather than bolting it on later.

### HANDS-ON BUILD — PHASE 1

Ship a grounded Q&A assistant with zero pipeline code: (1) drop a document corpus in S3 and attach a metadata sidecar per file; (2) create a customer-managed KB backed by OpenSearch Serverless with hierarchical chunking; (3) run ingestion; (4) query with `Retrieve` and inspect the chunks and scores, then with `RetrieveAndGenerate` for cited answers; (5) enable hybrid search + a reranker and re-test the same questions; (6) add a Guardrail with contextual grounding; (7) expose it via API Gateway + Lambda.

### EVAL CHECKPOINT

Assemble ~30 question/ideal-answer pairs from your corpus. Run **Bedrock Evaluations'** RAG evaluation — the "retrieval only" type to score whether the right chunks were fetched, and the retrieve-and-generate type to score answer quality and groundedness. Record the numbers; this is the **baseline every later phase must beat**. If managed KB already clears your bar, you may be done — that is a feature, not a failure.



### PHASE 1 TAKEAWAYS

- Know which KB product you're on: fully managed vs. customer-managed (all the knobs).
- **Chunking** is the highest-leverage setting; start hierarchical, go custom-Lambda for structured content.
- **OpenSearch Serverless** is the safe default store; **S3 Vectors** for cheap large corpora.
- **Retrieve** (you own generation) vs. **RetrieveAndGenerate** (managed loop) is an architecture choice.
- **Hybrid + reranking + metadata filtering** are the cheap, large relevance wins; filtering is also access control.
- Add **Guardrails** from the start; establish a **measured baseline** before building anything custom.

## PHASE 2

# The Retrieval Core, Decomposed

The managed KB hid five decisions behind one API. To go custom — or to tune the KB well — you must open the box. This phase is the retrieval engine: embeddings, stores, indexing, hybrid, reranking, and query understanding, each as a knob you control.

## 2.1 Embeddings: the choice that sets your ceiling

An embedding model maps text to a vector; retrieval quality can never exceed what your embeddings can distinguish. Three decisions:

- **Model.** On AWS, Titan Text Embeddings and Cohere Embed via Bedrock are the managed defaults; self-hosting an open model (e.g. a strong open embedder on SageMaker) buys control and can cut cost at volume. Match the model to your domain and languages.
- **Dimensionality.** Larger vectors capture more nuance but cost more to store and search. Some models support shortening dimensions with minimal quality loss — a direct cost lever.
- **Symmetry.** Query and document are often embedded the same way, but asymmetric setups (a query-tuned encoder) can help. Always embed queries and documents with the *same* model version; a version mismatch silently wrecks recall.

### FAILURE MODE

Re-embedding your corpus with a new model but forgetting to re-embed at query time (or vice versa) produces vectors from two incompatible spaces. Retrieval degrades to near-random and it looks like a ranking bug. Treat the embedding model + version as a pinned, corpus-wide invariant.

## 2.2 Vector stores on AWS — how to actually choose

Phase 1 listed the options; here is the engineering lens for picking one:

| Store                                                       | Strengths                                                  | Watch-outs                                                |
|-------------------------------------------------------------|------------------------------------------------------------|-----------------------------------------------------------|
| <b>OpenSearch Serverless</b>                                | Native hybrid (vector+BM25), filtering, mature ops, scales | Cost at idle; tune capacity units                         |
| <b>Aurora pgvector</b>                                      | Vectors + relational joins + transactions in one DB        | Index tuning is manual; scale limits vs. dedicated stores |
| <b>S3 Vectors</b>                                           | Lowest cost, serverless, huge corpora                      | Higher query latency; newer, fewer knobs                  |
| <b>Self-hosted (Qdrant / Weaviate / Milvus) on ECS/ EKS</b> | Full control, advanced filtering, no per-call markup       | You run it: HA, backups, upgrades, scaling                |

Rule of thumb: OpenSearch Serverless unless a specific force overrides it — pgvector when vectors live naturally beside relational data, S3 Vectors when the bill dominates, self-hosted when you need control the managed options can't give and have the team to operate it.

## 2.3 ANN indexing & HNSW

You cannot compare a query against millions of vectors one by one at query time — that is a brute-force scan. **Approximate Nearest Neighbour (ANN)** indexes trade a sliver of accuracy for enormous speed. The dominant algorithm is **HNSW** (Hierarchical Navigable Small World), a layered graph of vectors you descend to find near neighbours quickly. Three parameters you will tune:

| Parameter              | Controls                       | Trade-off                              |
|------------------------|--------------------------------|----------------------------------------|
| <b>M</b>               | Graph connections per node     | Higher = better recall, more memory    |
| <b>ef_construction</b> | Effort when building the index | Higher = better index, slower to build |
| <b>ef_search</b>       | Effort per query               | Higher = better recall, slower query   |

The practical loop: raise `ef_search` until recall on your eval set plateaus, then stop — further increases only cost latency. This single tuning pass often recovers "missing" documents people wrongly blame on embeddings.

## 2.4 Hybrid search, mechanically

Dense vectors match meaning but blur exact tokens; sparse keyword (BM25) nails exact terms, IDs, error codes, and rare names but misses paraphrase. **Hybrid** runs both and fuses the rankings — commonly with **Reciprocal Rank Fusion (RRF)**, which scores each document by the sum of  $1/(k + \text{rank})$  across the two lists, rewarding items that rank well in either. You tune the blend weight. On real enterprise corpora hybrid is close to a free lunch; make it the default, not an experiment.

## 2.5 Reranking

First-stage retrieval (vector/hybrid) optimises for recall at speed — cast a wide net. A **reranker** is a slower, more accurate cross-encoder that reads the query and each candidate *together* and scores true relevance, letting you keep only the best few for the context window. The pattern is universal:

query → retrieve top-50 (fast, hybrid) → RERANK (cross-encoder) → keep top-5 → into the prompt  
▲ recall-oriented ▲ precision-oriented ▲ token-budget-oriented

On AWS use Cohere Rerank or Amazon Rerank via Bedrock, or self-host a reranker on SageMaker. Retrieve wide + rerank narrow is often the highest quality-per-dollar change you can make.

## 2.6 Query understanding

The best retrieval on a badly-formed query still fails. A few cheap pre-retrieval transforms pay off:

- **Query rewriting / expansion.** Use a small model to fix, clarify, or expand the raw query (add synonyms, resolve "it").
- **Decomposition.** Split a compound question into sub-queries, retrieve for each, merge. This is the seed of agentic RAG (Phase 4).

- **HyDE (Hypothetical Document Embeddings).** Have the model draft a hypothetical answer, embed *that*, and retrieve against it — often closer to real documents than the bare question.
- **Conversational rewriting.** In chat, rewrite a follow-up ("what about its pricing?") into a standalone query using the history before retrieving.

#### HANDS-ON BUILD — PHASE 2

Rebuild Phase 1's retrieval as components you control: (1) embed the same corpus with a Bedrock embedder into an OpenSearch index you configure (set `M`, `ef_construction`); (2) implement hybrid vector+BM25 with RRF; (3) sweep `ef_search` and record recall vs. latency; (4) add a reranker over top-50 → top-5; (5) add query rewriting. Compare retrieval metrics against the Phase 1 baseline at each step.

#### EVAL CHECKPOINT

Score retrieval in isolation with **recall@k** (did the right chunk make the top-k?), **MRR** (how high did the first correct chunk rank?), and **nDCG** (graded ranking quality). Attribute each gain to a specific change — hybrid vs. rerank vs. query rewrite — so you keep only what earns its complexity.

#### PHASE 2 TAKEAWAYS

- Embeddings set your quality ceiling; pin model + version corpus-wide, embed query and doc identically.
- **OpenSearch Serverless** default; pgvector for relational co-location; S3 Vectors for cost; self-host for control.
- **HNSW** makes search fast; tune `ef_search` to the recall plateau.
- **Hybrid** (vector + BM25 via RRF) is a near-free win; make it default.
- **Retrieve wide, rerank narrow** is the top quality-per-dollar lever.
- Cheap **query rewriting / decomposition / HyDE** fix failures that no index tuning can.

## PHASE 3

# Custom RAG on ECS

This is the build-it-yourself phase — the shape most in-house platform teams converge on. You trade managed convenience for total control over parsing, chunking, indexing, retrieval, and cost. Only take this step when Phase 0's ladder pushed you here.

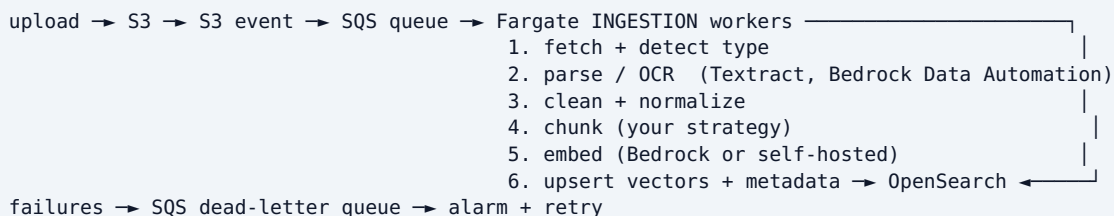
## 3.1 Why go custom (revisited, concretely)

Justify the custom build against specific, non-negotiable forces — not a general wish for control:

- **Custom parsing/chunking** the managed strategies destroy (code, transcripts, tables, mixed layouts).
- **Self-hosted models** for cost, latency, privacy, or fine-tuning you control.
- **Cost at high sustained volume**, where per-call managed markup dominates the bill.
- **Strict multi-tenancy, residency, or audit** requirements needing explicit isolation.
- **Bespoke retrieval logic** (multi-index federation, custom fusion, business rules) the KB can't express.

## 3.2 The ingestion pipeline is the hard part

Newcomers over-focus on query-time retrieval; in production, **ingestion is where most of the engineering and most of the failures live**. Treat it as a first-class, asynchronous, resumable pipeline — not a script. The stages:



Why SQS in front: it decouples bursty uploads from bounded worker capacity, gives you retries and a dead-letter queue for poison documents, and lets you scale workers on queue depth. Why Fargate: no servers to manage, and each document's parse/embed work is a natural containerised task.

## 3.3 Parsing & OCR — the quality bottleneck

Garbage parsing caps every downstream metric. Match the tool to the input:

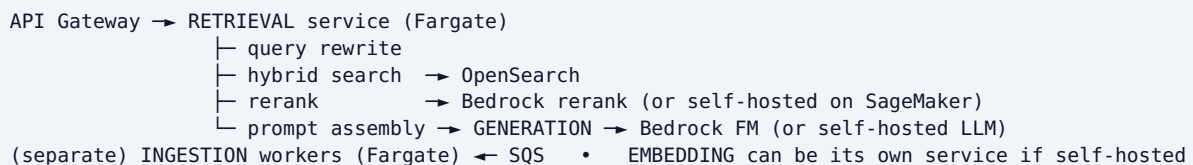
| Input                                | Tool                           | Note                                                   |
|--------------------------------------|--------------------------------|--------------------------------------------------------|
| Clean text / HTML / Markdown         | Library parsers                | Preserve structure (headings, lists) for chunking      |
| Scanned PDFs, images                 | <b>Amazon Textract</b>         | OCR + layout; extracts tables and forms, not just text |
| Complex mixed media (tables, charts) | <b>Bedrock Data Automation</b> | Structured extraction from documents/images/video      |
| Domain formats (code, transcripts)   | Custom parser                  | Chunk on semantic units (functions, turns)             |

#### CARRY STRUCTURE THROUGH

Retrieval quality is decided at parse time: keep headings, section paths, page numbers, and table boundaries as metadata on each chunk. That metadata powers filtering, citation, and the parent-context trick, and it cannot be recovered later if you flatten the document to raw text now.

### 3.4 The service topology

Split responsibilities into small ECS services behind an internal boundary, so each scales and deploys independently:



Keep **retrieval** and **ingestion** as separate services with separate scaling policies: ingestion is bursty and throughput-bound; retrieval is latency-bound and steady. If you self-host embeddings or the LLM, isolate those on GPU capacity (SageMaker endpoints or GPU ECS) so their scaling doesn't couple to the stateless services.

### 3.5 Orchestration: Fargate, SQS, Step Functions

- **SQS** for the ingestion work queue — buffering, retries, dead-letter isolation.
- **Step Functions** for multi-stage jobs with visibility — a full re-index, a batch evaluation run, or a document lifecycle with human-review branches. Use it when you need a visible, restartable state machine rather than fire-and-forget queue work.
- **ECS Fargate** for stateless compute — ingestion workers and retrieval/generation services — with auto-scaling on queue depth (ingestion) or request/latency (serving).
- **EventBridge** to trigger re-ingestion on source changes and to schedule periodic re-indexing.

### 3.6 Serving & scaling

Retrieval services are stateless, so scale horizontally behind an ALB/API Gateway with health checks. The usual bottlenecks are the vector store (size its capacity to your QPS and recall targets) and the generation model (rate limits on Bedrock, or GPU saturation if self-

hosted). Add a **semantic cache** (Phase 7.5) in front of generation to absorb repeat and near-duplicate queries. Set explicit timeouts and fallbacks at every hop — a slow reranker should degrade to un-reranked results, not a 30-second hang.

### 3.7 Infrastructure as code

A pipeline of this many moving parts is unmaintainable click-by-click. Define everything — queues, services, index, IAM roles, alarms — in **CDK**, **Terraform**, or CloudFormation, so environments are reproducible and reviewable. This also makes the Phase 7 CI/CD eval gate possible: infra and eval config live beside the code.

#### HANDS-ON BUILD — PHASE 3

Stand up the full custom pipeline: (1) S3→event→SQS→Fargate ingestion workers doing parse (Textract for scanned docs) → chunk → Bedrock embed → OpenSearch upsert, with a dead-letter queue; (2) a Fargate retrieval service doing query-rewrite → hybrid → rerank → Bedrock generation behind API Gateway; (3) Step Functions for a full re-index job; (4) define it all in CDK/Terraform; (5) auto-scale ingestion on queue depth and retrieval on latency.

#### EVAL CHECKPOINT

Re-run the Phase 1 question set end-to-end. You must now **beat the managed baseline** on retrieval metrics or answer quality — otherwise the custom complexity isn't paying for itself and you should reconsider. Also measure ingestion throughput (docs/min) and the dead-letter rate; a rising DLQ is your earliest signal of parsing problems.

#### PHASE 3 TAKEAWAYS

- Go custom only for a concrete force: custom parsing, self-hosted models, cost at scale, strict tenancy, bespoke logic.
- **Ingestion** — async via SQS+Fargate with a dead-letter queue — is the hard, failure-prone part.
- **Parsing quality** (Textract / Bedrock Data Automation) caps every downstream metric; carry structure through as metadata.
- Separate **ingestion** (throughput) and **retrieval** (latency) services with independent scaling.
- **Step Functions** for visible multi-stage jobs; **IaC** for everything.
- The custom build must **beat the managed baseline** or it isn't earning its complexity.

## PHASE 4

# Agentic RAG

Everything so far did one retrieve-then-generate pass. Agentic RAG makes retrieval a *tool the model decides how to use* — planning, retrieving multiple times, calling other tools, and correcting itself. It is powerful and expensive; adopt it only when a simpler baseline provably falls short.

## 4.1 When retrieval becomes a tool

In classic RAG, retrieval is a fixed step that always runs, once, before generation. In agentic RAG, the model sits in a loop and *chooses*: does this question even need retrieval? From which source? Should I decompose it? Was what I got enough, or should I search again? This shift — from a fixed pipeline to a **plan-act-observe** loop — is the whole idea.

```
classic:  query → retrieve → generate → answer
agentic:  query → [ plan → act (retrieve / SQL / graph / web) → observe → reflect ]* → answer
                ▲──────────────────────────────────┘
                loop until confident
```

### ADOPT DELIBERATELY

Each loop iteration is another model call — more latency, more tokens, more cost, more ways to fail. Agentic RAG earns its keep on questions a single pass cannot answer: multi-hop, multi-source, or tool-requiring. If your queries are simple lookups, an agent is over-engineering. Justify it with the Phase 1 eval numbers, not enthusiasm.

## 4.2 The patterns

"Agentic RAG" is an umbrella. These are the concrete, nameable patterns you will actually build, roughly in order of sophistication:

| Pattern                              | What it adds                                                          | Use when                                |
|--------------------------------------|-----------------------------------------------------------------------|-----------------------------------------|
| <b>Router</b>                        | Model picks which index/tool fits the query                           | Multiple distinct sources               |
| <b>Query planner / decomposition</b> | Breaks a compound question into sub-queries, retrieves each           | Multi-part questions                    |
| <b>Multi-hop</b>                     | Uses one retrieval's result to form the next query                    | Answers that chain across documents     |
| <b>Corrective RAG (CRAG)</b>         | Grades retrieved docs; re-retrieves or falls back to web if weak      | Recall is unreliable                    |
| <b>Self-RAG</b>                      | Model decides <i>whether</i> to retrieve and critiques its own output | Mixed queries; avoid needless retrieval |
| <b>ReAct</b>                         | Interleaves reasoning and tool actions in a loop                      | General tool-using agents               |



## 4.3 The AWS agent stack (it changed — read this)

The AWS agent landscape shifted significantly, and older tutorials are now misleading:

- **Bedrock Agents is now "Bedrock Agents Classic"** and closes to new customers on **July 30, 2026**. Do not start new work on it. AWS directs you to AgentCore instead.
- **Amazon Bedrock AgentCore** (GA October 2025) is the production layer for agents: a composable set of services — **Runtime** (serverless, session-isolated agent hosting), **Gateway** (turns APIs/Lambdas into agent tools via MCP), **Memory**, **Identity** (IAM/Cognito auth), **Observability** (CloudWatch), plus **Code Interpreter** and **Browser** tools. It is framework-agnostic and model-agnostic, with VPC, PrivateLink, and CloudFormation support.
- **Strands Agents** is AWS's open-source agent SDK. You write the agent logic (tools, loop, multi-agent handoffs) and deploy it on AgentCore Runtime with minimal glue. It supports multi-agent primitives (handoffs, swarm, graph workflows) and the A2A protocol.

### THE PRODUCTION SHAPE

Write the agent in **Strands** (or LangGraph/CrewAI — AgentCore is framework-agnostic), expose your RAG retrievers and other systems as tools through **AgentCore Gateway** using **MCP**, run it on **AgentCore Runtime**, and get memory, identity, and tracing from AgentCore's other services. This is the current-generation replacement for the old Bedrock Agents flow.

## 4.4 Designing good tools

An agent is only as good as its tools and their descriptions — the model routes by reading them. Principles:

- **One clear job per tool.** "search\_policy\_docs" and "query\_sales\_db" beat one vague "search". Split into focused tools; do not build a mega-tool (or a mega-agent).
- **Descriptions are prompts.** Write the tool's name, description, and parameters as carefully as a prompt — they are what the model uses to decide when to call it.
- **Return structured, bounded results.** Give the agent clean, trimmed context (top-k reranked chunks + provenance), not raw dumps that blow the context budget.
- **Protect systems of record.** Prefer querying a vector index or read replica over letting an agent hit production databases directly.

**MCP** (Model Context Protocol) is the standard interface for exposing these tools so they are reusable across agents; **A2A** (Agent-to-Agent) is the protocol for specialised agents to call one another.

## 4.5 Memory & multi-agent

- **Memory.** AgentCore Memory persists conversation and task state so an agent keeps continuity across turns and sessions. Distinguish **short-term** (this conversation) from **long-term** (durable user/task facts) and store only what you need — memory is context cost and a privacy surface.
- **Multi-agent.** For complex workflows, split into specialised agents (a researcher, a SQL analyst, a writer) coordinated by handoffs or a supervisor, rather than one agent juggling everything. Split when a single agent's tool list or prompt grows unwieldy — but remember every agent is more latency and more failure surface.

## 4.6 The agentic RAG failure surface

Agents inherit and *amplify* every limitation of the underlying model (see the LLM guide's limits part). Three to design against from day one:

- **Compounding errors.** A small per-step error rate multiplied over many steps yields an unreliable whole. Keep loops short and add explicit stop conditions.
- **Prompt injection.** When the agent reads untrusted content (a web page, a retrieved doc, a tool result), hidden instructions there can hijack it. Treat all tool output as untrusted data, never as instructions; constrain tool permissions tightly.
- **Cost/latency blow-ups.** An agent that loops can quietly make dozens of model calls. Cap iterations, budget tokens, and monitor per-request cost.

### HANDS-ON BUILD — PHASE 4

Build an agentic RAG over multiple sources: (1) write a Strands agent with three tools — a vector-KB search (your Phase 3 retriever), a second index, and web search — each with a crisp description; (2) implement a router + query decomposition so compound questions fan out and merge; (3) add a CRAG step that grades retrieved chunks and re-retrieves or web-falls-back when they're weak; (4) deploy on AgentCore Runtime with Memory and Observability; (5) cap loop iterations and token budget.

### EVAL CHECKPOINT

Build a **multi-hop** eval set — questions a single retrieval can't answer — and compare the agent against the Phase 3 single-pass system. The agent should win clearly *on those questions*; if it also wins on simple ones, you're paying latency for nothing — add a Self-RAG gate so the agent skips retrieval when it isn't needed. Track cost-per-answer and average loop count alongside accuracy.

### PHASE 4 TAKEAWAYS

- Agentic RAG makes retrieval a **tool in a plan-act-observe loop**; adopt it only for multi-hop/multi-source/tool questions.
- Know the patterns: **router, planner, multi-hop, CRAG, Self-RAG, ReAct**.
- The AWS stack changed: **Bedrock Agents is Classic (sunsetting)** — use **AgentCore + Strands + MCP**.
- Tools with **clear, well-described single jobs** make or break routing; return bounded, structured context.
- Design against **compounding errors, prompt injection, and cost blow-ups** from the start.
- Evaluate on **multi-hop** questions and watch cost-per-answer, not just accuracy.

# GraphRAG: Knowledge Graph as Source

Vector search finds text that is *similar*. Some questions instead need text that is *connected* — across entities and relationships, often several hops apart. GraphRAG retrieves over a knowledge graph, alone or fused with vectors, to answer those.

## 5.1 When graphs win

Reach for a graph when the *relationships* are the point, not just the content:

- **Multi-hop / connect-the-dots** questions: "Which vendors supply components used in products affected by this recall?" — the answer chains across linked entities no single chunk contains.
- **Relationship-centric domains**: supply chains, org and ownership structures, dependency graphs, legal/citation networks, medical ontologies, and **data/metadata lineage**.
- **Global-summary** questions: "What are the main themes across this whole corpus?" — graph community structure summarises better than any top-k of chunks.
- **Explainability**: a traversal path is auditable provenance — you can show *why* two facts are connected.

### THE HONEST CAVEAT

For plain "find the passage that answers this" questions, a graph is overkill — vector or hybrid retrieval is simpler and just as good. Graphs earn their construction and maintenance cost only when connectivity is genuinely central to the questions. Don't graph everything.

## 5.2 Managed GraphRAG (Bedrock KB + Neptune Analytics)

The fastest path: Bedrock Knowledge Bases ships a fully managed **GraphRAG** (GA March 2025) backed by **Neptune Analytics**. It automatically builds a graph of entities and relationships from your documents, stores vector embeddings alongside it, and at query time **combines vector similarity search with graph traversal** — giving more complete, explainable answers to multi-step questions without you writing any graph code. Know its limits before you commit:

| Consideration      | Managed GraphRAG today                                                               |
|--------------------|--------------------------------------------------------------------------------------|
| Data source        | Amazon S3 only                                                                       |
| Graph construction | Automatic; uses Claude 3 Haiku; <b>not customizable</b>                              |
| Scale              | ~1,000 files per data source (raisable); Neptune Analytics autoscaling not supported |
| Control            | You cannot tune the graph schema or extraction                                       |

Verdict: excellent when the automatic graph fits your data and you value speed over control. When you need a specific schema, custom extraction, or sources beyond S3, build it yourself (5.3).

## 5.3 Custom graph construction

A custom GraphRAG has two builds: constructing the graph (offline) and retrieving over it (online). Construction:

```
docs → chunk → ENTITY + RELATION EXTRACTION (LLM with a schema prompt)
           e.g. (Company)-[:SUPPLIES]->(Component)-[:USED_IN]->(Product)
           → resolve/dedupe entities (the same "Acme Corp" everywhere)
           → load nodes + edges + properties → Neptune
           → also embed chunks → vector index (for hybrid entry points)
```

The hard parts are **schema design** (what entity and relation types matter for your questions) and **entity resolution** (merging aliases and duplicates so the graph is connected rather than fragmented). Both reward domain knowledge; get the schema right by working backward from the questions you must answer.

## 5.4 Neptune & traversal

**Amazon Neptune** is the managed graph database; **Neptune Analytics** adds fast in-memory analytics and vector support (and is what managed GraphRAG uses). You query with **openCypher** or **Gremlin** — traversals like "from this entity, follow SUPPLIES then USED\_IN up to 3 hops, filtered by relation type." Keep hop counts bounded (1-3); traversals can explode combinatorially, and beyond a few hops relevance usually decays faster than useful signal grows.

## 5.5 Hybrid graph + vector retrieval

The strongest custom pattern uses vectors to *enter* the graph and traversal to *expand*:

```
query → vector search (k=10) → find entry chunks → extract their entity IDs
      → Neptune traversal (1-3 hops from those entities, filtered)
      → assemble context = top chunks + subgraph facts + provenance path
      → generate
```

Vector search handles "what is this question about"; the graph handles "what is connected to it." The provenance path doubles as an explanation and a citation. This hybrid is usually better than either pure vector or pure graph retrieval.

### HANDS-ON BUILD — PHASE 5

Two tracks. **Managed:** point a Bedrock KB at an S3 corpus with linked entities, enable GraphRAG on Neptune Analytics, and compare its multi-hop answers to your Phase 2 vector baseline. **Custom:** (1) define a small entity/relation schema for your domain; (2) run LLM extraction over chunks; (3) resolve entities and load Neptune; (4) implement the vector-entry → graph-expand retrieval with bounded hops and provenance; (5) expose it as an agent tool from Phase 4.

### EVAL CHECKPOINT

GraphRAG's value shows only on the right questions, so build a **multi-hop, relationship-centric** eval set and a **global-summary** set. Compare graph, vector, and hybrid retrieval on each. Expect graph/hybrid to win the connection questions and vector to tie on simple lookups — confirming you should *route* question types to the right retriever rather than forcing everything through the graph.

### PHASE 5 TAKEAWAYS

- Use graphs when **relationships and multi-hop connections** are the question — not for plain passage lookup.
- **Managed GraphRAG** (Bedrock KB + Neptune Analytics) is fast but S3-only, auto-built, and not customizable.
- **Custom** construction hinges on **schema design** and **entity resolution**, worked backward from your questions.
- Query Neptune with **openCypher/Gremlin**; keep traversals to **1-3 hops**.
- **Hybrid graph + vector** (vector to enter, traversal to expand) is the strongest pattern, with the path as provenance.
- Route question types to the right retriever; don't force everything through the graph.

## Structured & Multimodal RAG

Not all knowledge is prose. A huge share of enterprise answers live in *tables* (warehouses, databases) and in *media* (scanned forms, charts, images). Forcing these through a text-chunk pipeline throws away structure; this phase retrieves from them natively.

### 6.1 Text-to-SQL: querying tables in natural language

When the answer is an aggregate over rows — "total refunds by region last quarter" — vector search is the wrong tool; you need a database query. **Text-to-SQL** retrieval translates a natural-language question into SQL, runs it, and feeds the result to the model. Bedrock Knowledge Bases supports this natively over **Amazon Redshift** and **SageMaker Lakehouse** (and you can build it custom over Athena/Glue). The model needs the **schema** (tables, columns, types, and good descriptions) as context to generate correct SQL — schema quality is the main accuracy lever.

#### GUARDRAIL

Generated SQL is untrusted. Run it against a **read-only** role, on a replica, with row/column permissions and query limits — never let a text-to-SQL path reach a writable production connection. Validate and constrain the SQL before execution.

### 6.2 When "documents" are tables

Even inside PDFs, tables carry meaning that flattening to text destroys (row/column relationships, headers). Extract them structurally — Textract's table extraction or Bedrock Data Automation — and preserve the structure: index a row's cells with its column headers as metadata, or serialise each row as a small structured record. This lets "what was the value for X in year Y" retrieve the exact cell rather than a mangled paragraph.

### 6.3 Multimodal parsing & retrieval

Charts, diagrams, scanned forms, and product images hold answers that pure-text pipelines miss. *Bedrock Data Automation and vision foundation models analyze tables, charts, and images within documents, keeping source references to the visual content.* Two approaches:

- **Describe-then-index.** Use a vision model to generate a rich text description of each image/chart, embed the description, and retrieve it like any chunk — simple and works with your existing text stack.
- **Multimodal embeddings.** Embed images and text into a shared vector space so a text query can retrieve a relevant image directly. More powerful for image-heavy corpora.

Either way, keep a link back to the source visual so answers can cite and display it.

### 6.4 Federating structured + unstructured

Real questions cross modalities: "Summarise complaints about product X and show its return rate." That needs unstructured retrieval (complaints) *and* a SQL aggregate (return rate). The clean way to federate is the Phase 4 agent: expose each retriever — vector KB,

text-to-SQL, graph — as a separate tool and let the agent route and combine. Structured/multimodal RAG is therefore less a separate architecture than a set of **additional tools** your agent can reach for.

#### HANDS-ON BUILD — PHASE 6

(1) Stand up text-to-SQL over a Redshift/Glue table with a well-described schema and a read-only role; (2) extract tables from a set of PDFs with Textract and index rows with header metadata; (3) add describe-then-index for any charts/images; (4) wire all three as tools into your Phase 4 agent so a single question can pull from prose, tables, and media together.

#### EVAL CHECKPOINT

For text-to-SQL, measure **execution accuracy** (does the generated query return the correct result?), not text similarity. For table/multimodal retrieval, check that the exact cell or figure is retrieved and cited. For federated questions, verify the agent used the *right* tool for each part.

#### PHASE 6 TAKEAWAYS

- **Text-to-SQL** (Bedrock KB over Redshift/Lakehouse) answers aggregate questions vectors can't; schema quality drives accuracy; run read-only.
- Preserve **table structure** at extraction; index rows with header metadata for exact-cell retrieval.
- **Multimodal**: describe-then-index (simple) or shared multimodal embeddings (powerful); always keep a source link.
- Federate modalities by exposing each as an **agent tool** (Phase 4), not a monolith.

---

## PHASE 7

# Production Hardening

A RAG demo and a RAG product differ almost entirely in this phase. Evaluation, observability, security, cost, and latency are what separate "it worked in the notebook" from "it holds up under real traffic." This is where most of the durable engineering value lives.

## 7.1 Evaluation — the two halves

You cannot improve what you don't measure, and RAG has **two** things to measure separately, because a good answer needs both to work:

| Half       | Question                                 | Metrics                                                  |
|------------|------------------------------------------|----------------------------------------------------------|
| Retrieval  | Did we fetch the right context?          | recall@k, MRR, nDCG, context precision/recall            |
| Generation | Given that context, was the answer good? | faithfulness/groundedness, answer relevance, correctness |

**Faithfulness (groundedness)** is the RAG-specific one: is every claim in the answer supported by the retrieved context, or did the model add unsupported material (hallucinate)? Frameworks like **RAGAS** compute these; **Bedrock Evaluations** provides managed RAG evaluation (retrieval-only and retrieve-and-generate) using an LLM-as-judge. Always separate the halves: a bad answer from good context is a generation/prompt problem; a bad answer from bad context is a retrieval problem — and the fixes are entirely different.

## 7.2 Continuous evaluation in CI/CD

Run evaluation as a **gate in the pipeline**, not a one-off notebook. Keep a versioned "golden" question/answer set. On every change — a new chunking strategy, prompt, model, or index setting — run the eval automatically and **block the deploy if scores regress**. This turns quality from a vibe into a tracked, defended metric, and catches the silent regressions (a prompt tweak that quietly drops faithfulness) that otherwise reach users.

## 7.3 Observability & cost

RAG is a multi-hop distributed system; when an answer is wrong you must see *which* hop failed.

- **Tracing.** Emit an OpenTelemetry trace per request spanning query-rewrite → retrieve → rerank → generate (and each agent loop), so you can inspect the retrieved chunks and the exact prompt for any bad answer. AgentCore Observability and CloudWatch surface this for agents.
- **Token & cost tracking.** Log input/output tokens and cost per request, per tenant, per feature. A per-query cost dashboard turns runaway agent loops and prompt bloat into visible line items before they become a bill.
- **Quality signals.** Capture thumbs-up/down, citation click-through, and "no answer found" rates as online proxies for quality between offline eval runs.



## 7.4 Security & multi-tenancy

- **Access control at retrieval.** Enforce permissions by **filtering before you retrieve** (metadata: tenant, role, classification). Never retrieve broadly and filter the answer afterward — the model has already seen data it shouldn't.
- **Isolation.** For multi-tenant systems choose an isolation model deliberately: separate indexes/collections per tenant (strong isolation, more overhead) vs. a shared index with mandatory tenant filters (efficient, but one missing filter is a leak). Match the choice to your compliance bar.
- **PII & guardrails.** Redact or block sensitive data at ingestion and with Bedrock Guardrails at query time; log access for audit.
- **Network & identity.** Keep traffic in-VPC (PrivateLink to Bedrock), scope every service with least-privilege IAM, and encrypt at rest and in transit.
- **Prompt injection.** Treat all retrieved and tool content as untrusted data, not instructions — especially for agents that act on external text.

## 7.5 Caching

Two caches cut cost and latency materially. A **semantic cache** stores answers keyed by query *embedding*, so a paraphrase of an earlier question returns instantly without retrieval or generation — set a similarity threshold and a TTL to avoid stale hits. **Prompt caching** (supported by Bedrock for repeated context prefixes) avoids re-processing a large, stable system prompt or document on every call. Both are high-ROI once traffic has repetition.

## 7.6 Latency & scaling

Budget latency per hop and attack the biggest: generation usually dominates, so stream tokens to the user, keep the reranked context tight, and cache. Retrieval scales with your vector store's provisioned capacity; generation scales with Bedrock rate limits or your GPU fleet. Set timeouts and graceful fallbacks everywhere — degrade to un-reranked results or a smaller model rather than failing — and load-test to find the knee before your users do.

### HANDS-ON BUILD — PHASE 7

Harden the Phase 3/4 system: (1) build a golden eval set and a RAGAS/Bedrock-Evaluations harness scoring retrieval and generation separately; (2) wire it as a CI/CD gate that blocks regressions; (3) add OpenTelemetry tracing across all hops and a per-query cost dashboard; (4) enforce tenant metadata filtering and a Guardrail; (5) add a semantic cache with a similarity threshold and TTL; (6) load-test and add per-hop timeouts and fallbacks.

### EVAL CHECKPOINT

The system now has a **living scoreboard**: retrieval and generation metrics tracked on every change, cost-per-query and p95 latency on dashboards, and a deploy gate that enforces them. That scoreboard — not any single architecture — is what makes the system productionised.

### PHASE 7 TAKEAWAYS

- Evaluate **retrieval and generation separately**; **faithfulness/groundedness** is the RAG-specific metric.
- Run eval as a **CI/CD gate** against a golden set to block silent regressions.
- **Trace every hop** and track **token cost** per request/tenant/feature.
- Enforce access by **filtering before retrieval**; choose a tenant isolation model deliberately; keep traffic in-VPC with least-privilege IAM.
- **Semantic + prompt caching** cut cost/latency; set timeouts and fallbacks on every hop.
- A **living scoreboard**, not a clever architecture, is what "production" means.

---

## PHASE 8

# Capstone & Reference Material

The curriculum ends where real work begins: a scenario, a justified architecture, and a built system with a scoreboard. This phase is the capstone brief plus the reference tables you'll keep open while building.

## 8.1 The capstone brief

Pick a real scenario (ideally from your own domain) and take it end to end. The deliverable is not just a working system but a **defensible architecture decision**:

1. **State the requirements** on the three axes: source shape, query complexity, and the forces (volume, latency, tenancy, compliance, budget).
2. **Walk the Phase 0 decision ladder** and choose the least-complex pattern that meets them. Write down *why* you stopped where you did — and what would push you one rung higher.
3. **Build the baseline first** (Phase 1) and measure it. This is your yardstick.
4. **Add exactly the sophistication the requirements justify** — custom pipeline, agent, graph, structured/multimodal — and prove each addition beats the baseline on the relevant eval set.
5. **Harden it** (Phase 7): separated eval in CI/CD, tracing, cost dashboard, security, caching.
6. **Write the one-page decision record**: the requirements, the options considered, the choice, the trade-offs accepted, and the measured results.

## 8.2 A worked decision (example)

### SCENARIO → ARCHITECTURE

*"An internal assistant over 5,000 policy PDFs and a warehouse of HR records. Questions are mostly direct lookups but some cross both ('what's the parental-leave policy and my remaining balance?'). ~50 users, strict per-employee data isolation."*

**Walk the ladder:** standard docs + single-shot → managed KB handles the policies (rung 1). But some questions cross into structured HR data and combine sources (rung 4), and strict per-employee isolation demands enforced filtering. **Decision:** a Phase 4 agent with two tools — a Bedrock KB over the policy PDFs (hierarchical chunking, hybrid, reranked) and a text-to-SQL tool over the HR warehouse (read-only, row-level filtered by employee) — with mandatory tenant/identity filtering, Guardrails, and separated eval. **Not** custom ECS (volume is low), **not** a graph (no multi-hop relationship questions). Least complexity that meets every requirement.

## 8.3 Service cheat-sheet

| Job                                       | AWS service                                                           | Phase |
|-------------------------------------------|-----------------------------------------------------------------------|-------|
| Managed RAG (chunk→retrieve)              | Bedrock Knowledge Bases                                               | 1     |
| Foundation & embedding models             | Amazon Bedrock                                                        | 1-2   |
| Vector store (default)                    | OpenSearch Serverless                                                 | 1-2   |
| Vector store (cheap / relational / graph) | S3 Vectors / Aurora pgvector / Neptune Analytics                      | 2, 5  |
| Reranking                                 | Cohere Rerank / Amazon Rerank (via Bedrock)                           | 1-2   |
| Parsing & OCR                             | Amazon Textract; Bedrock Data Automation                              | 3, 6  |
| Ingestion queue / workers                 | SQS + ECS Fargate                                                     | 3     |
| Pipeline orchestration                    | Step Functions; EventBridge                                           | 3     |
| Self-hosted models (embed / LLM / rerank) | SageMaker; GPU ECS                                                    | 2-3   |
| Agent hosting & services                  | Bedrock AgentCore (Runtime, Gateway, Memory, Identity, Observability) | 4     |
| Agent SDK / tools protocol                | Strands Agents; MCP; A2A                                              | 4     |
| Graph database                            | Amazon Neptune / Neptune Analytics                                    | 5     |
| Structured data (text-to-SQL)             | Redshift; SageMaker Lakehouse; Athena/Glue                            | 6     |
| Evaluation                                | Bedrock Evaluations; RAGAS                                            | 7     |
| Safety                                    | Bedrock Guardrails                                                    | 1, 7  |
| Observability / cost                      | CloudWatch; OpenTelemetry                                             | 7     |
| API & auth                                | API Gateway; Lambda; IAM; Cognito                                     | 1, 3  |
| Infrastructure as code                    | AWS CDK / Terraform / CloudFormation                                  | 3-7   |

## 8.4 Glossary

| Term                                  | Meaning                                                                                  |
|---------------------------------------|------------------------------------------------------------------------------------------|
| <b>Knowledge Base (KB)</b>            | Bedrock's managed RAG service; two products (fully managed, customer-managed).           |
| <b>Retrieve / RetrieveAndGenerate</b> | KB APIs: return chunks only, vs. return a cited answer.                                  |
| <b>Chunking</b>                       | Splitting docs into retrieval units: fixed, semantic, hierarchical, custom.              |
| <b>Embedding</b>                      | Vector representation of text; retrieval matches by vector similarity.                   |
| <b>ANN / HNSW</b>                     | Approximate nearest-neighbour search; HNSW is the common graph index (M, ef_search).     |
| <b>Hybrid search</b>                  | Fusing dense vector + sparse keyword (BM25), often via RRF.                              |
| <b>RRF</b>                            | Reciprocal Rank Fusion: combines rankings by summing $1/(k+rank)$ .                      |
| <b>Reranking</b>                      | Cross-encoder re-scoring of candidates for precision before generation.                  |
| <b>Metadata filtering</b>             | Restricting retrieval by attributes; also the access-control lever.                      |
| <b>HyDE</b>                           | Embedding a hypothetical answer to retrieve closer to real documents.                    |
| <b>Fargate</b>                        | Serverless containers (ECS) for ingestion workers and services.                          |
| <b>Textract</b>                       | AWS OCR + layout/table/form extraction from scanned docs.                                |
| <b>Bedrock Data Automation</b>        | Structured extraction from documents, images, and video.                                 |
| <b>Agentic RAG</b>                    | Retrieval as a tool inside a plan-act-observe loop.                                      |
| <b>CRAG / Self-RAG</b>                | Grade-and-re-retrieve; decide-whether-to-retrieve + self-critique.                       |
| <b>ReAct</b>                          | Interleaving reasoning and tool actions in a loop.                                       |
| <b>AgentCore</b>                      | Bedrock's production agent platform (Runtime, Gateway, Memory, Identity, Observability). |
| <b>Strands</b>                        | AWS's open-source agent SDK; deploys on AgentCore.                                       |
| <b>MCP / A2A</b>                      | Protocols: tools-to-agent (MCP), agent-to-agent (A2A).                                   |
| <b>GraphRAG</b>                       | Retrieval over an entity/relationship graph, often fused with vectors.                   |
| <b>Neptune / Neptune Analytics</b>    | AWS graph DB; Analytics adds in-memory + vectors (powers managed GraphRAG).              |
| <b>openCypher / Gremlin</b>           | Graph query languages for traversals.                                                    |
| <b>Entity resolution</b>              | Merging aliases/duplicates so the graph is connected.                                    |
| <b>Text-to-SQL</b>                    | Translating natural language into SQL over a warehouse.                                  |
| <b>Faithfulness / groundedness</b>    | Whether every answer claim is supported by retrieved context.                            |

|                              |                                                                         |
|------------------------------|-------------------------------------------------------------------------|
| <b>recall@k / MRR / nDCG</b> | Retrieval metrics: found in top-k / rank of first hit / graded ranking. |
| <b>RAGAS</b>                 | Framework for RAG eval (faithfulness, relevance, context metrics).      |
| <b>Semantic cache</b>        | Caching answers keyed by query embedding to skip repeat work.           |
| <b>Guardrails</b>            | Bedrock content/PII/grounding safety filters.                           |

#### WHERE THIS SITS IN THE SERIES

The **RAG Study Guide** taught RAG in the abstract; the **LLM Study Guide** taught the model inside it; this guide turns both into **production systems on AWS**. Together they span concept to deployment. The discipline to carry forward from all three: understand the machine, pick the simplest design that meets the requirement, and defend it with measurement.